

Class 2 Assignment — One-Page Summary

Jonas Altrock, Niklas Brandtner, Denis Vakorin

1. The original problems (OrderProcessing.java)

- **Global mutable state** — `public static List<Order> orders`: any code can mutate the "database", `ArrayList` is not thread-safe, and tests poison each other because state cannot be reset.
- **God method** — `processOrder` mixed five concerns: logging, validation, simulated slow work (`Thread.sleep(1000)`), persistence, and status signaling.
- **Ad-hoc error handling** — `System.out.println / e.printStackTrace()` (two channels), `InterruptedException` swallowed and processing *continues*; failure reported as a bare `boolean` with no reason.
- **Anemic domain object** — `Order` had public mutable fields, validated only *outside* itself (invalid orders were constructible), `double money`, no `equals/hashCode`.
- **Leaky internals** — `getAllOrders()` returned the live list; duplicate ids accepted; $O(n)$ lookup; two classes in one file, no package, zero tests.

2. The redesigned architecture

```
Main (composition root – wires concrete objects into abstractions)
└─ OrderService.processOrder(id, amount) → OrderProcessingResult
    └─ Order                immutable value object, validates ITSELF
    └─ OrderRepositoryInterface ← InMemoryOrderRepository (HashMap, O(1))
    └─ ExternalOrderSystem     ← SlowExternalOrderSystem (1 s, behind seam)
    └─ Logger (java.util.logging) cross-cutting concern, injected
```

- **Modules & responsibilities** — `Order`: domain invariants; `OrderService`: orchestration only; `OrderRepositoryInterface`: persistence contract; `ExternalOrderSystem`: slow external dependency; `OrderProcessingResult`: explicit outcome (`INVALID` | `DUPLICATE` | `TIMEOUT` | `SAVED`).
- **Information hiding** — the service sees only interfaces (DIP); repository internals (`HashMap`) are private; `findAll()` returns a defensive copy. Storage, external system, and logging are swappable without touching business logic (OCP).
- **Maintainability** — each module has one reason to change; extending (e.g., DB repository, new validation) means adding an implementation, not editing `OrderService`.

3. The refactored Java code (src manual refactor/swe/class2assignment)

File	Responsibility
Order.java	final fields, self-validating (<code>validate()</code> throws <code>IllegalArgumentException</code>),

File	Responsibility
	toString
OrderService.java	validate → duplicate check → external call → save, returns result enum
OrderProcessingResult.java	four explicit outcomes instead of boolean
OrderRepositoryInterface.java	find/findAll/save/delete/clear contract
ExternalOrderSystem.java	interface with process(Order) throws InterruptedException
test/ InMemoryOrderRepository.java	HashMap store: O(1) find, duplicate-free, defensive copies
test/ SlowExternalOrderSystem.java	simulated slow dependency, out of the business path
Main.java	composition root; wires logger, fake repo, fake external system; asserts SAVED then DUPLICATE

Core excerpt:

```
public OrderProcessingResult processOrder(String id, double amount) {
    Order order = new Order(id, amount);
    if (!order.validate()) return OrderProcessingResult.INVALID;
    if (orderRepository.find(id) != null) return
OrderProcessingResult.DUPLICATE;
    try { externalOrderSystem.process(order); }
    catch (InterruptedException e) { return OrderProcessingResult.TIMEOUT; }
    orderRepository.save(order);
    return OrderProcessingResult.SAVED;
}
```

4. The craftsmanship improvements

- **Predictable** — outcomes are data (enum), not an ambiguous boolean; expected failures and programming errors are separated.
- **Testable** — constructor injection, no statics; a fake repository and fast fake external system make tests instant and isolated.
- **Honest gaps (to fix)** — `Order.validate()` throws instead of returning false, so the `INVALID` branch is currently unreachable (service should catch the exception or validate must not throw for expected invalid input); interrupt flag is not restored after `InterruptedException`; amount is still double (should be `BigDecimal`); no real build tool/test suite yet.

5. The integration benefits

- **Continuous integration** — no global state ⇒ tests are deterministic and run in any order/parallel; the 1 s sleep lives only in a fake, so CI is fast; `Main`'s assertions are the seed

of an automated smoke test that a build step (javac/JUnit) can fail on.

- **Safe evolution** — interfaces are stable contracts: swapping the in-memory repository for a DB or the fake external system for a real client requires **zero** changes to `OrderService` (low coupling, high cohesion $\Rightarrow$ low blast radius per change).
- **Low technical debt & API stability** — every module has a single reason to change; failures are explicit values, so new outcomes extend the enum without breaking callers' compilation in unclear ways; information hiding keeps internal representation free to evolve.